

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
ABES ENGINEERING COLLEGE, GHAZIABAD

PROJECT SYNOPSIS

Software Requirements Specification (SRS)

*Project Title: Loan Default Prediction with Explainable AI,
Drift Monitoring and Real-Time Scoring*
Standard Academic Compliance: IEEE Std 830-1998

SUBMITTED BY:

1. Vaishnavi Dutt (Roll No: _____)

Degree: Bachelor of Technology (B.Tech)

Academic Year: 2026 - 2027

Programme: HCL Training Project

UNDER THE GUIDANCE OF:

Prof. / Dr. _____

Designation: _____

Department of Computer Science & Engg.

Status: 3rd Year Project Synopsis

Date of Submission: ____ - ____ - 2026

Document Revision & Approval History

Version	Date	Description / Major Changes	Prepared By	Reviewed By
0.1	28-Sep-2026	First synopsis: problem statement, objectives, proposed stack and Lending Club dataset	Vaishnavi Dutt	Project Guide
0.2	30-Sep-2026	Whole stack moved to Python (FastAPI + Streamlit) as advised by the trainer; dataset switched to the Kaggle Credit Risk Dataset	Vaishnavi Dutt	Project Guide
1.0	01-Oct-2026	Rewritten in IEEE 830 SRS format with real results of Steps 1-7 (model, NPA simulation, SHAP, API)	Vaishnavi Dutt	Project Guide

Guidance Note: This synopsis follows the IEEE 830-1998 recommended practice for Software Requirements Specifications. All numbers quoted in this document (dataset size, ROC-AUC, NPA reduction, latency) are measured results from the project code, not estimates. Items marked “Planned” are not yet built.

1. Introduction

1.1 Purpose

The purpose of this document is to give a complete, clear and verifiable description of the system **‘Loan Default Prediction with Explainable AI, Drift Monitoring and Real-Time Scoring’**: what it does, who uses it, its external interfaces, its performance targets and its constraints. It serves as the reference agreement between the student developer, the project guide, the HCL trainer and the evaluators.

Abstract. Banks lose money when borrowers do not repay their loans; such loans become Non-Performing Assets (NPAs). Manual credit checks are slow and inconsistent, and many machine-learning models are ‘black boxes’ that a loan officer cannot justify to a customer or a regulator. This project builds an end-to-end credit-risk system in Python. A calibrated XGBoost model, trained on 32,409 cleaned loan records, predicts the probability that an applicant will default and reaches a **test ROC-AUC of 0.952** (target ≥ 0.85). Every decision (APPROVE / REVIEW / REJECT) is explained with the top five SHAP reasons. In a simulation on unseen test loans, rejecting only the high-risk band cuts defaulted loans by **75.8%** and defaulted money by **79.8%** compared with approving everyone (target: 30% NPA reduction). The model is served through a FastAPI backend with JWT login, a Streamlit frontend and a PostgreSQL database (measured scoring time about 112 ms), and a drift monitor with automatic retraining keeps the model reliable over time.

1.2 Scope of the System

Problem statement. Given the details a borrower provides at application time (age, income, home ownership, employment length, loan purpose, loan grade, amount, interest rate, credit history), predict whether the loan will default, explain why, and do it fast enough to be used while the customer is waiting.

Objectives.

Obj ID	Objective	Status
O1	Clean and analyse a real loan dataset (EDA) without data leakage	Done
O2	Train and compare Logistic Regression, Random Forest and XGBoost (with and without SMOTE)	Done
O3	Reach ROC-AUC ≥ 0.85 on unseen test data	Done – 0.952
O4	Turn probabilities into APPROVE / REVIEW / REJECT and show $\geq 30\%$ NPA reduction in simulation	Done – 75.8% / 79.8%
O5	Explain every prediction with SHAP (top 5 reasons in plain English)	Done
O6	Real-time scoring REST API with login, roles and audit log in PostgreSQL	Done
O7	Web frontend for loan officers and an admin dashboard	In progress
O8	Data-drift monitoring (Evidently + PSI) with alert and safe retraining	Planned

Obj ID	Objective	Status
O9	Docker packaging, CI tests, final report and presentation	Planned

- **In-Scope Capabilities:** data cleaning and EDA, model training with cross-validation and tuning, probability calibration, decision bands, SHAP explanations, secure REST API, Streamlit web app, PostgreSQL storage of applications/predictions/audit log, drift detection and retraining, automated tests.
- **Out-of-Scope Boundaries:** real bank integration (core banking, credit bureau APIs), real customer data, loan disbursement and repayment tracking, mobile apps, cloud production deployment and regulatory certification.
- **Expected Benefits:** faster and consistent credit decisions (about 0.1 s per application), fewer bad loans (75.8% fewer defaults in simulation), transparent reasons for every decision, and early warning when customer behaviour changes.

1.3 Definitions, Acronyms, and Abbreviations

Category	Term / Acronym	Definition / Context
Standard	SRS	Software Requirements Specification (IEEE Std 830-1998)
Domain	Default	Borrower fails to repay the loan (target loan_status = 1)
Domain	NPA	Non-Performing Asset: a loan on which repayment has stopped; here, a defaulted loan
Domain	Loan grade	Lender's risk grade A (safest) to G (riskiest)
ML	ROC-AUC	Area under the ROC curve; 0.5 = random guess, 1.0 = perfect ranking of defaulters above non-defaulters
ML	PR-AUC / KS	Precision-recall area and Kolmogorov-Smirnov statistic; credit-risk metrics for imbalanced data
ML	SMOTE	Synthetic Minority Over-sampling Technique for class imbalance
ML	Calibration	Adjusting scores so that 0.30 really means about a 30% chance of default (isotonic regression)
ML	Data leakage	Using information that is only known after the loan is given; produces fake high accuracy
XAI	SHAP	SHapley Additive exPlanations: contribution of each feature to one prediction
MLOps	Data drift / PSI	Change in input data over time; Population Stability Index measures it (> 0.2 = significant)

Category	Term / Acronym	Definition / Context
MLOps	MLflow	Tool that tracks experiments and stores versioned models (model registry)
Security	JWT / RBAC	JSON Web Token for login sessions; Role-Based Access Control (admin, officer)
Interface	API / REST	Application Programming Interface over HTTP with JSON messages
Analysis	EDA	Exploratory Data Analysis: charts and statistics before modelling

1.4 References

- **IEEE Std 830-1998:** IEEE Recommended Practice for Software Requirements Specifications.
- **Sommerville, I.:** Software Engineering, 10th Edition, Pearson Education, 2016.
- **Chen, T. & Guestrin, C. (2016):** XGBoost: A Scalable Tree Boosting System, Proc. ACM KDD.
- **Lundberg, S. & Lee, S.-I. (2017):** A Unified Approach to Interpreting Model Predictions (SHAP), NeurIPS.
- **Breiman, L. (2001):** Random Forests, Machine Learning 45(1).
- **Chawla, N. et al. (2002):** SMOTE: Synthetic Minority Over-sampling Technique, JAIR 16.
- **Niculescu-Mizil, A. & Caruana, R. (2005):** Predicting Good Probabilities with Supervised Learning, ICML.
- **Pedregosa, F. et al. (2011):** Scikit-learn: Machine Learning in Python, JMLR 12.
- **Dataset:** Credit Risk Dataset (laotse), Kaggle – <https://www.kaggle.com/datasets/laotse/credit-risk-dataset>
- **Tools documentation:** FastAPI, Streamlit, SQLAlchemy, MLflow, Evidently AI, Optuna (official docs).
- **Project repository:** <https://github.com/ExploringVD/Loan-Defalut-Prediction>

1.5 Document Overview

Section 2 describes the product context, users, environment, constraints and the dataset. Section 3 lists the functional requirements module by module. Section 4 covers the external interfaces, Section 5 the non-functional requirements, and Section 6 the design models (architecture, use case, ER diagram). Section 7 presents the methodology and results achieved so far, Section 8 the project plan, Section 9 the conclusion and future scope, and Section 10 the academic sign-off.

2. Overall Description

2.1 Product Perspective

The system is a self-contained, multi-tier web application with two parts. The **online part** scores loan applications in real time: a loan officer fills a form in the Streamlit frontend, the FastAPI backend validates it, runs the saved model, calculates SHAP reasons, stores everything in PostgreSQL and returns the decision. The **offline part** trains, tunes and registers the model, and the drift monitor regularly compares new applications with the training data and triggers retraining when the data has changed.

Online system (real-time scoring)

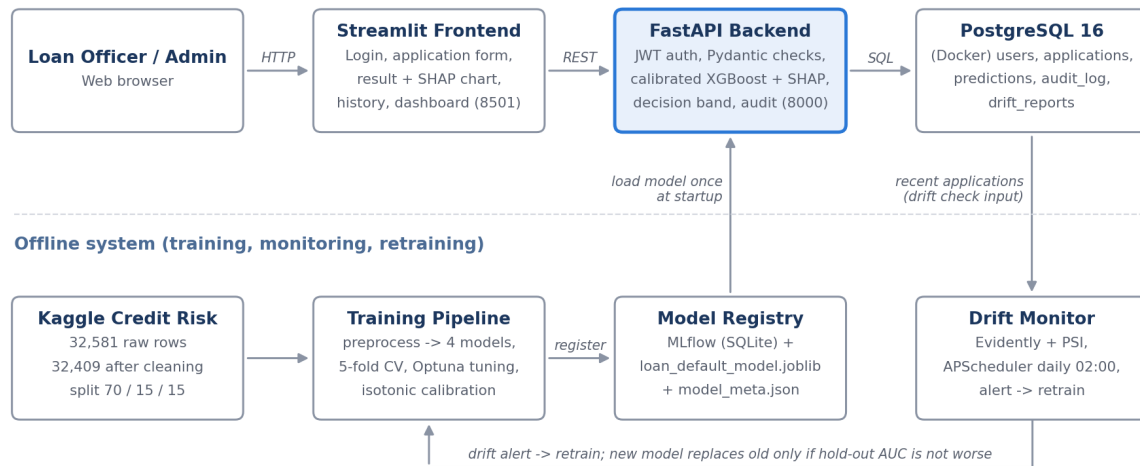


Figure 1: System architecture (online scoring and offline training / monitoring)

2.2 User Classes and Characteristics

User Role / Persona	Technical Proficiency	Core Responsibilities & Rights
Administrator	High	Manages users, views the dashboard (decision counts, latency, model version, drift status), runs drift checks and retraining
Loan Officer	Medium (non-technical)	Logs in, submits applications, reads the risk score, decision and top 5 reasons, sees own application history
Scheduler / Data Scientist	High (automated job)	Runs the daily drift check, retrains the model and replaces it only if the new model is not worse

2.3 Operating Environment

- Client Requirements:** any modern web browser (Chrome, Safari, Firefox, Edge); frontend at <http://localhost:8501>.

- **Development / Server Machine:** MacBook Air (Apple Silicon), 8 GB RAM, macOS; Python 3.12 in a virtual environment (uv); VS Code and JupyterLab.
- **Database Storage Tier:** PostgreSQL 16 running in a Docker container (memory limited to 512 MB); SQLite only for automated tests.
- **Model Storage:** MLflow model registry (local SQLite backend) and a saved joblib pipeline in the models/ folder.

2.4 Design and Implementation Constraints

- **Language Constraint:** everything (ML, backend, frontend) must be written in Python, as required by the trainer.
- **Hardware Constraint:** 8 GB RAM, so training uses `n_jobs = 2`, at most 50 Optuna trials, and only the needed Docker containers run.
- **No Data Leakage:** only fields known at application time are used; all imputers, encoders and SMOTE are fitted on training data only; the test split is used once.
- **Security Compliance:** passwords stored only as bcrypt hashes; JWT tokens (HS256) expire after 60 minutes; secrets kept in a `.env` file that is never pushed to GitHub.
- **Reproducibility:** `random_state = 42` everywhere; one saved pipeline (preprocessing + model) used by training, API and drift checks.

2.5 Assumptions and Dependencies

- **Assumptions:** applicants enter honest values; the Kaggle data is representative of the lender's customers; the dataset has no application dates, so drift is demonstrated with simulated new batches (higher interest rates, lower incomes, more debt-consolidation and medical loans).
- **Dependencies:** open-source Python libraries (pandas, scikit-learn, XGBoost, SHAP, FastAPI, Streamlit, SQLAlchemy, MLflow, Evidently), Docker Desktop for PostgreSQL, and GitHub for version control.

2.6 Dataset Description

The **Credit Risk Dataset** (Kaggle, laotse) contains 32,581 loans and 12 columns. After cleaning, 32,409 rows remain (165 duplicates, 5 rows with age above 100 and 2 rows with impossible employment length removed). 21.9% of loans defaulted, so the data is imbalanced. Missing values: `loan_int_rate` 9.5%, `person_emp_length` 2.7% (filled inside the model pipeline). Split: 70% train (22,685), 15% validation (4,862), 15% test (4,862), stratified.

Field	Meaning	Type
<code>person_age</code>	Age of the applicant	Numeric
<code>person_income</code>	Annual income	Numeric
<code>person_home_ownership</code>	RENT / OWN / MORTGAGE / OTHER	Categorical
<code>person_emp_length</code>	Years in current employment	Numeric (missing 2.7%)
<code>loan_intent</code>	Purpose: education, medical, venture, personal, home improvement, debt consolidation	Categorical

Field	Meaning	Type
loan_grade	Risk grade A to G (also used as number 1-7)	Ordinal
loan_amnt	Loan amount requested	Numeric
loan_int_rate	Interest rate offered	Numeric (missing 9.5%)
loan_percent_income	Loan amount / annual income	Numeric
cb_person_default_on_file	Defaulted before (Y / N)	Binary
cb_person_cred_hist_length	Length of credit history in years	Numeric
loan_status	Target: 1 = default, 0 = repaid	Binary target

Key EDA findings. Default rate rises from 10.0% (grade A) to 98.4% (grade G), with a sharp jump from grade C (20.8%) to D (59.0%). Loans larger than 30% of income default 70.4% of the time versus 15.4%. Renters default 31.6%, mortgage holders 12.6% and owners 7.5%. The poorest fifth of applicants default 43.3% versus 9.2% for the richest fifth.

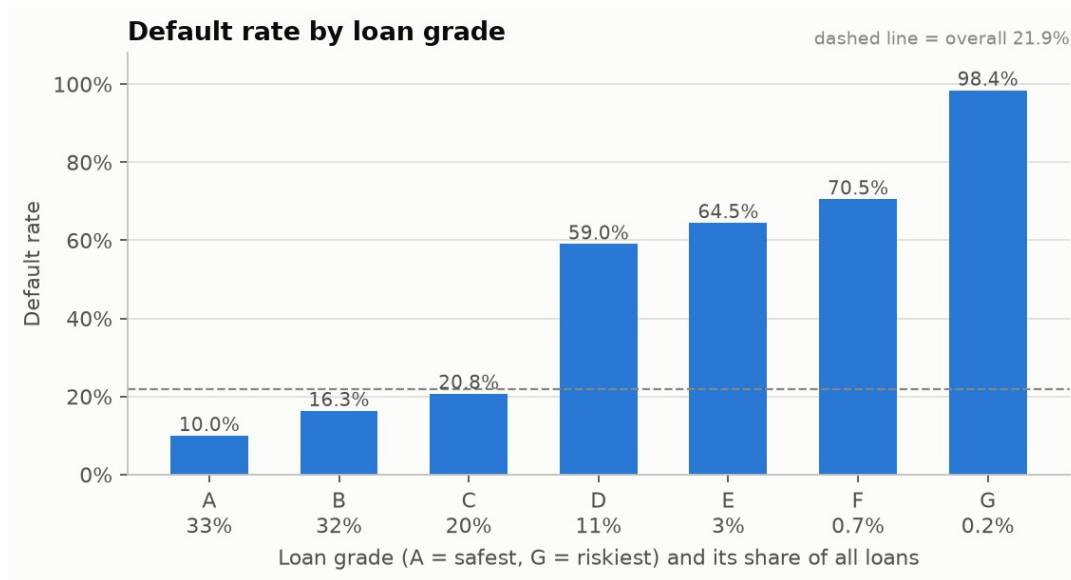


Figure 2: Default rate by loan grade (EDA)

3. System Features and Functional Requirements

Requirements are grouped by module. Each has a unique ID used for testing and traceability. Priority: High = must have for the demo, Medium = should have, Low = nice to have.

3.1 Module 1: Authentication and Access Control

Req ID	Feature Description	Input / Validation Rule	Priority
FR-1.1	User login	Username + password (OAuth2 form); bcrypt check; returns JWT valid for 60 minutes	High
FR-1.2	Role-based access	Roles admin and officer; admin-only endpoints return 403 for officers	High
FR-1.3	Current user details	GET /auth/me with valid bearer token; 401 if token missing or expired	Medium
FR-1.4	Audit logging	Every login, scoring and admin action written to audit_log with user and time	High
FR-1.5	Seed default users	One admin and one officer created at first start from .env values	Low

3.2 Module 2: Real-Time Scoring and Explanation

Req ID	Feature Description	Input / Validation Rule	Priority
FR-2.1	Submit loan application	11 applicant fields + applicant name; Pydantic checks ranges (e.g. age 18-100, income > 0, grade A-G)	High
FR-2.2	Predict probability of default	Saved calibrated XGBoost pipeline loaded once at start-up; output between 0 and 1	High
FR-2.3	Decision band	APPROVE if $p < 0.10$, REVIEW if $0.10 \leq p < 0.35$, REJECT if $p \geq 0.35$ (cut-offs stored in model_meta.json)	High
FR-2.4	Top 5 reasons	SHAP values converted to plain-English reasons (e.g. "Loan is 45% of income – raises risk")	High
FR-2.5	Store result	Application, prediction, model version and latency saved in PostgreSQL	High
FR-2.6	View past applications	GET /applications (own list for officer, all for admin) and GET /applications/{id}	Medium

3.3 Module 3: Model Training and Management

Req ID	Feature Description	Input / Validation Rule	Priority
FR-3.1	Data cleaning and split	Remove duplicates and impossible values; 70/15/15 stratified split, random_state 42	High
FR-3.2	Train and compare models	LR, RF, XGBoost, XGBoost + SMOTE; 5-fold stratified CV; ROC-AUC, PR-AUC, KS, F1	High
FR-3.3	Hyperparameter tuning	Optuna (50 trials) on CV ROC-AUC; isotonic probability calibration	High
FR-3.4	Model registry	Final model registered in MLflow (alias production) and saved as joblib + metadata JSON	High
FR-3.5	Model information	GET /model/info returns version, metrics, cut-offs and training date	Medium

3.4 Module 4: Drift Monitoring and Retraining (Planned)

Req ID	Feature Description	Input / Validation Rule	Priority
FR-4.1	Drift report	Evidently compares recent applications with training data; HTML report saved	High
FR-4.2	Prediction stability	PSI of predicted probabilities between training and recent data	High
FR-4.3	Drift alert	Alert when share of drifted features > 0.3 or PSI > 0.2; stored in drift_reports	High
FR-4.4	Scheduled check	APScheduler runs the check daily at 02:00; admin can also run it on demand	Medium
FR-4.5	Safe retraining	retrain.py trains a new model; it replaces production only if hold-out ROC-AUC is not worse	Medium

3.5 Module 5: Frontend, Dashboard and Reporting

Req ID	Feature Description	Input / Validation Rule	Priority
FR-5.1	Login page	Streamlit form; token kept in session; logout button	High

Req ID	Feature Description	Input / Validation Rule	Priority
FR-5.2	New application page	Form with dropdowns and limits; shows probability gauge, coloured decision and SHAP bar chart	High
FR-5.3	History page	Table of past applications with filters by decision	Medium
FR-5.4	Admin dashboard	Decision counts, average latency, model version, latest drift status (GET /monitoring/stats)	Medium
FR-5.5	Reports	Model comparison, business simulation and SHAP insight reports in Markdown with charts	Low

4. External Interface Requirements

4.1 User Interfaces (UI)

The Streamlit web app has a sidebar menu with four pages: **Login**, **New Application**, **History** and **Dashboard** (admin only). Decisions are colour coded (green APPROVE, amber REVIEW, red REJECT) and each result shows the probability, the decision and a SHAP bar chart of the top reasons. The backend also provides interactive API documentation (Swagger UI) at <http://localhost:8000/docs>, shown below.

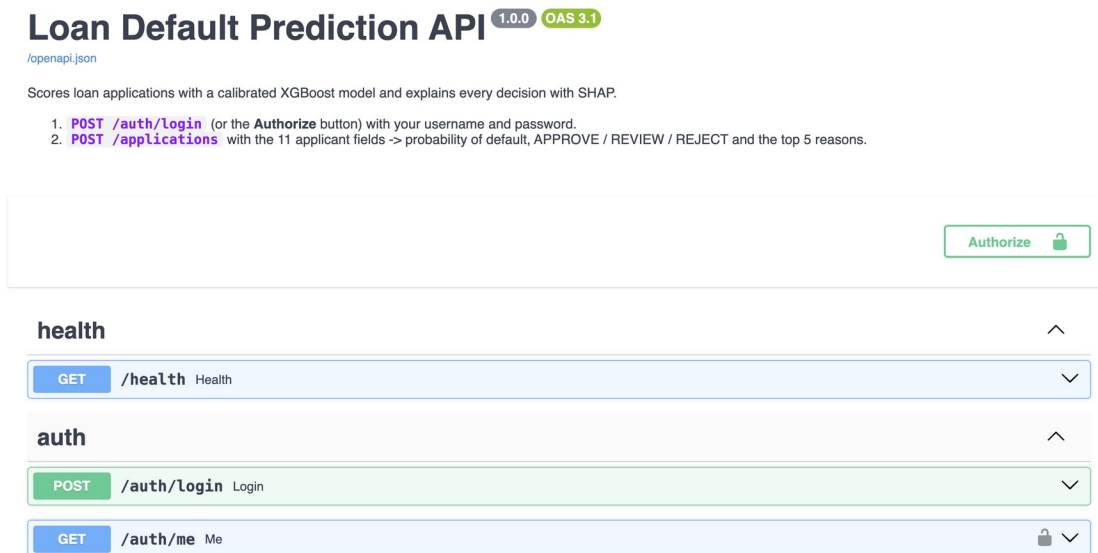


Figure 3: Swagger UI of the Loan Default Prediction API (FastAPI)

4.2 Hardware Interfaces

- **Standard Deployment:** laptop or desktop with keyboard, mouse and a display of at least 720p; no special hardware.
- **Minimum for running the full stack:** 8 GB RAM, about 5 GB free disk (Python environment, Docker image, data and models).

4.3 Software Interfaces

- **Database:** PostgreSQL 16 accessed through SQLAlchemy 2.0 ORM; schema created and versioned with Alembic migrations.
- **Model:** scikit-learn Pipeline (imputer + encoder + calibrated XGBoost) loaded with joblib; MLflow registry for versions.
- **Explainability:** SHAP TreeExplainer on the XGBoost model inside the pipeline.
- **Monitoring:** Evidently AI for data-drift reports; APScheduler for the daily job.

4.4 Communication Interfaces

- **Protocol:** HTTP REST between Streamlit (port 8501) and FastAPI (port 8000); HTTPS when deployed.
- **Data Serialization:** UTF-8 JSON request and response bodies validated with Pydantic schemas.
- **Authentication:** Authorization: Bearer <JWT> header on every protected endpoint.
- **Database Link:** PostgreSQL wire protocol on port 5432 to the Docker container (psycopg2 driver).

Method	Endpoint	Purpose	Access
GET	/health	Service and model status	Public
POST	/auth/login	Login, returns JWT	Public
GET	/auth/me	Current user	Logged in
POST	/applications	Score a new application	Officer / Admin
GET	/applications, /applications/{id}	List / view applications	Officer / Admin
GET	/model/info	Model version and metrics	Logged in
GET	/monitoring/stats	Dashboard statistics	Admin

5. Non-Functional Requirements (NFRs)

Attribute	Target Metric / Standard	Verification & Result
5.1 Performance	Scoring + explanation < 200 ms per application	Measured 112 ms scoring, 174 ms full request (x-process-time header)

Attribute	Target Metric / Standard	Verification & Result
5.2 Accuracy	Test ROC-AUC ≥ 0.85	0.952 on 4,862 unseen test loans (PR-AUC 0.909, KS 0.767)
5.3 Business impact	$\geq 30\%$ NPA reduction in simulation	75.8% fewer defaults, 79.8% less defaulted money (only REJECT turned away)
5.4 Security	Hashed passwords, token login, role checks	bcrypt + JWT (60 min); pytest checks 401/403 responses
5.5 Explainability	100% of decisions have reasons	Top 5 SHAP reasons stored with every prediction
5.6 Reliability	Calibrated, stable probabilities	Expected calibration error 0.004, Brier score 0.051 on test
5.7 Traceability	Every action can be audited	audit_log table + model_version on each prediction
5.8 Maintainability	Readable, tested, version-controlled code	Modular src/ and app/ folders, pytest suite, Git + GitHub
5.9 Portability	Runs on any machine with Python and Docker	requirements.txt, Docker Compose (planned for full stack)

6. System Design and Analysis Models

Diagram Type	Objective & Scope	Where
Architecture Diagram	Online scoring path and offline training / monitoring loop	Figure 1 (Section 2.1)
Use Case Diagram	Actors (Loan Officer, Admin, Scheduler) and what each can do	Figure 4
ER Diagram	Database tables, keys and relationships in PostgreSQL	Figure 5
ML Pipeline	Clean $\rightarrow$ split $\rightarrow$ preprocess $\rightarrow$ train $\rightarrow$ tune $\rightarrow$ calibrate $\rightarrow$ register	Section 7.1

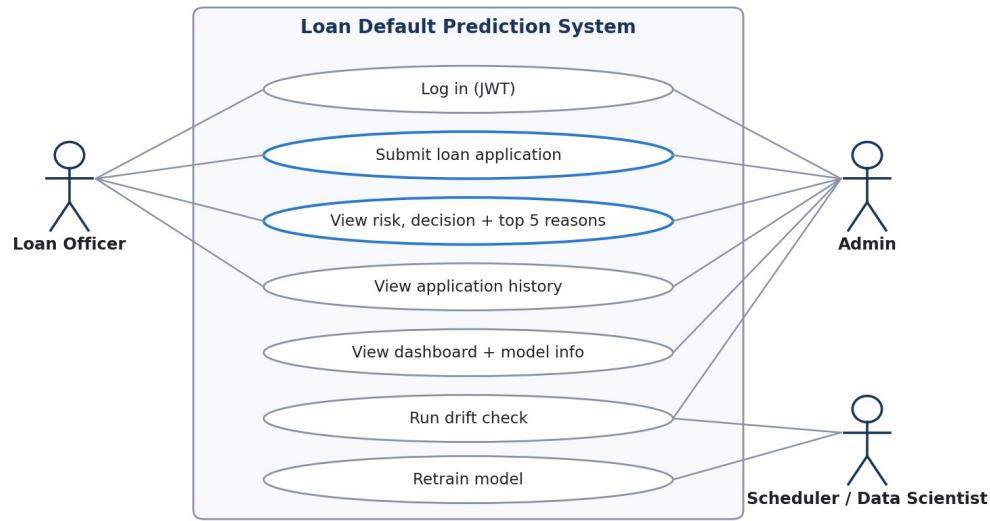
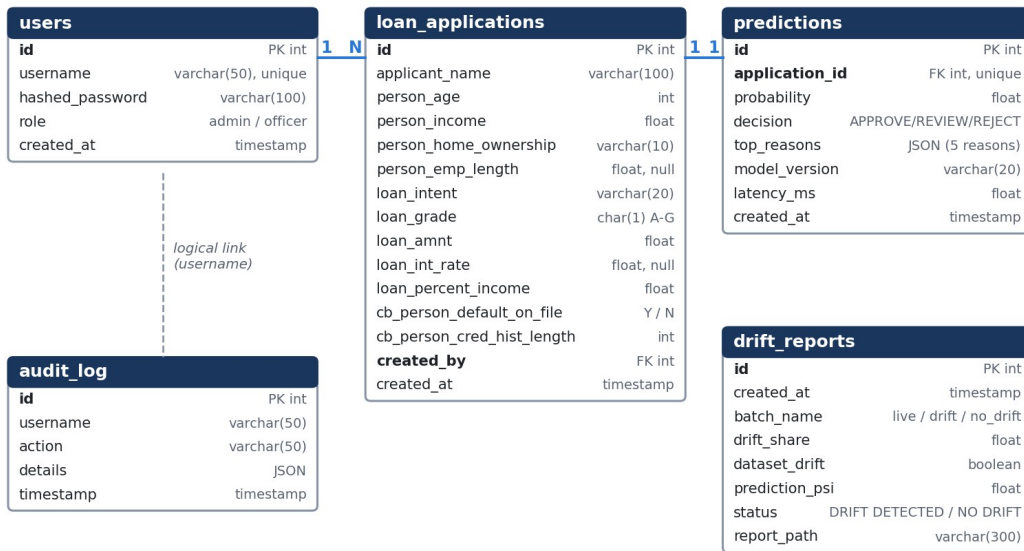


Figure 4: Use case diagram



drift_reports is independent: written by the drift monitor

Figure 5: Entity-relationship diagram of the PostgreSQL database

Relationships: one user creates many loan applications (1 : N); each application has exactly one prediction (1 : 1); audit_log is linked to users by username; drift_reports is written independently by the drift monitor.

7. Methodology and Results Achieved

7.1 Machine-Learning Pipeline

- **Step 1 – Cleaning:** duplicates and impossible rows removed; target and Y/N flags converted to 0/1; grade converted to a number 1-7.
- **Step 2 – Preprocessing:** scikit-learn ColumnTransformer: median imputation for numeric fields, one-hot encoding for categories, all inside one Pipeline fitted on training data only. A leakage check confirmed no column gives a suspiciously perfect score.
- **Step 3 – Model comparison:** four models trained with 5-fold stratified cross-validation; class imbalance handled with class weights / scale_pos_weight (3.57) or SMOTE.
- **Step 4 – Tuning and calibration:** Optuna (50 trials) tuned XGBoost (best CV ROC-AUC 0.946, train-validation gap reduced from 0.040 to 0.028); isotonic calibration lowered calibration error from 0.078 to 0.010; final model refit on train + validation and tested once.
- **Step 5 – Decision bands:** cut-offs chosen on validation data: REJECT ≥ 0.35 (turns away at most 2% of good borrowers), APPROVE < 0.10 (REVIEW + REJECT still hold $\geq 90\%$ of defaulters).
- **Step 6 – Explainability:** SHAP values for global importance and per-applicant reasons.
- **Step 7 – Serving:** FastAPI backend with PostgreSQL, JWT, audit log and automated tests.

7.2 Model Comparison (validation split)

Model	CV ROC-AUC	Train ROC-AUC	Val ROC-AUC	Val PR-AUC	F1 @0.5
XGBoost (selected)	0.942 $\pm$ 0.005	0.995	0.955	0.912	0.824
XGBoost + SMOTE	0.943 $\pm$ 0.004	0.991	0.952	0.912	0.844
Random Forest	0.930 $\pm$ 0.006	1.000	0.940	0.896	0.826
Logistic Regression	0.873 $\pm$ 0.005	0.873	0.879	0.726	0.635

Tree models beat Logistic Regression because default risk follows step-like patterns (grade C $\rightarrow$ D, loan above 30% of income, renters). SMOTE did not clearly help (+0.001 CV AUC, within one standard deviation), so plain XGBoost with scale_pos_weight was chosen because it is simpler and creates no synthetic loans.

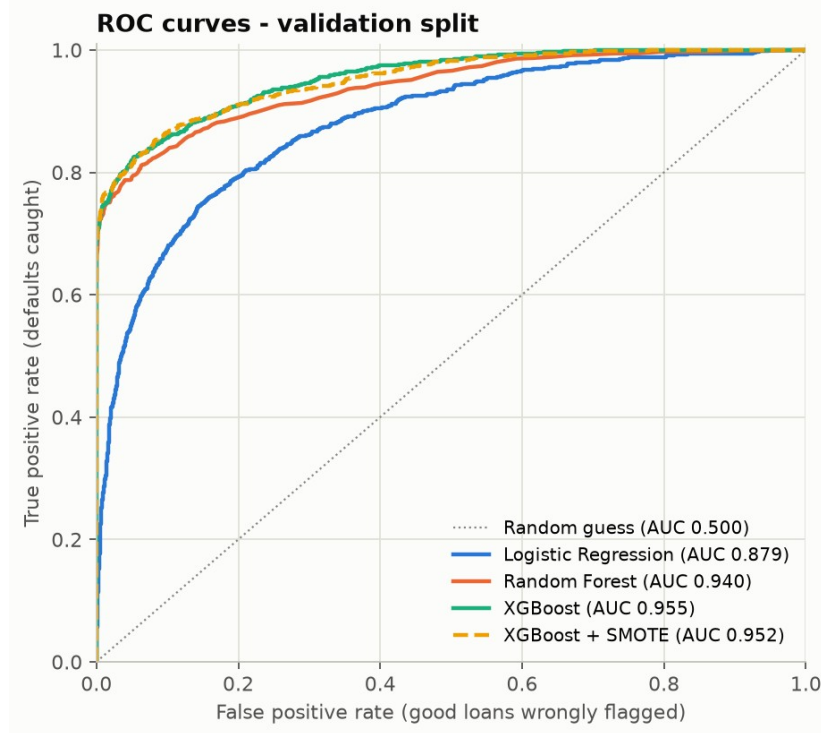


Figure 6: ROC curves of the four models on the validation split

7.3 Final Model on the Test Split (4,862 unseen loans)

Metric	Value	Meaning
ROC-AUC	0.952	Ranks a random defaulter above a random good borrower 95% of the time
PR-AUC	0.909	High precision and recall on the minority (default) class
KS statistic	0.767	Strong separation between defaulters and non-defaulters
Precision / Recall @0.5	0.975 / 0.726	Almost every flagged loan really defaults; 73% of defaulters flagged
F1 @0.5	0.832	Balance of precision and recall
Brier score / ECE	0.051 / 0.004	Probabilities are well calibrated

7.4 Business Simulation – NPA Reduction

Baseline = approve every test loan (what really happened): 1,063 of 4,862 loans defaulted, \$11.44 million lost.

Policy	Defaults approved	Defaulted amount	NPA reduction (count / amount)	Good loans turned away
Approve everyone	1,063	\$11,439,550	–	0
A: reject only REJECT band	257	\$2,308,875	75.8% / 79.8%	51 (1.3%)
B: reject REVIEW + REJECT	89	\$794,900	91.6% / 93.1%	868 (22.8%)

Both policies exceed the 30% target. Policy A is recommended: it removes most bad loans while losing only 1.3% of good customers, and REVIEW cases go to a human officer.

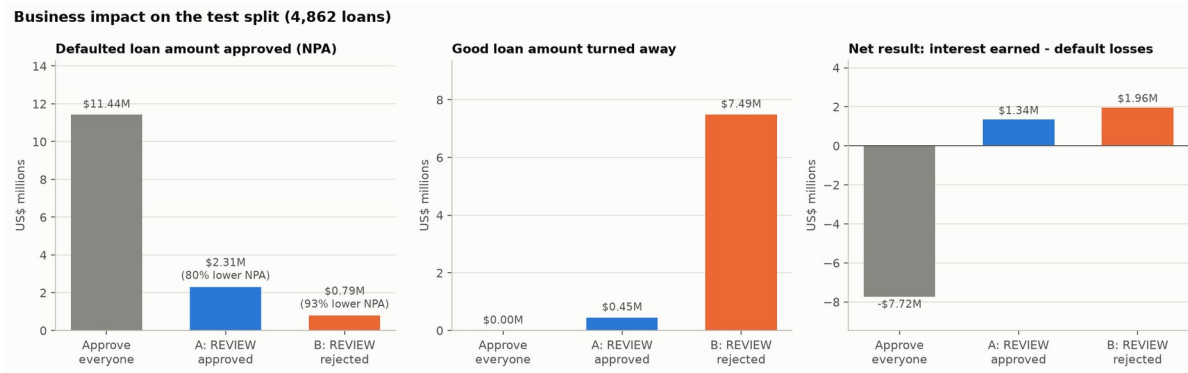


Figure 7: NPA reduction on the test split for both policies

7.5 Explainability (SHAP)

Most important features by mean |SHAP|: **income** (21%), **loan grade** (17%), **loan as % of income** (15%), **home ownership** (14%) and **loan purpose** (12%), followed by interest rate, amount, employment length, age and credit history. Previous default adds little because that information is already inside the loan grade. For each applicant the API returns the five features that pushed the risk up or down the most, in plain English.

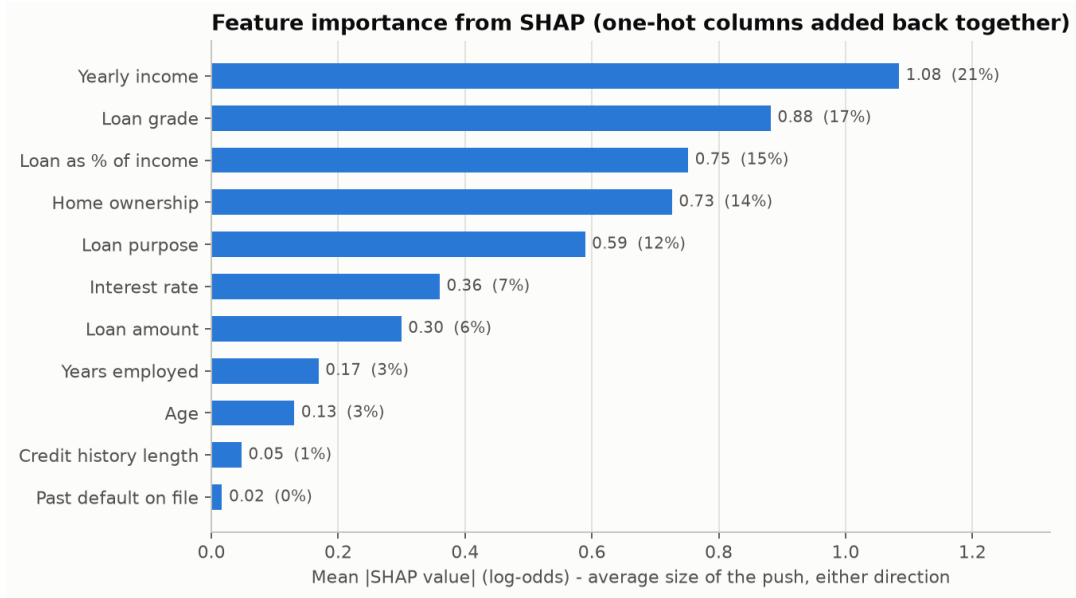


Figure 8: Global feature importance (mean absolute SHAP value)

7.6 Honest Limitations

- **Very clean rule in the data:** all 1,650 renters whose loan exceeded 30% of income defaulted, which suggests partly rule-generated labels; real-bank AUC would likely be lower. (An earlier Lending Club version reached only about 0.70 AUC without leakage.)
- **Baseline is “approve everyone”:** a real bank already screens applicants, so real NPA reduction would be smaller.
- **No dates in the data:** drift is shown with simulated batches rather than real time-based change.

8. Project Plan and Current Status

Step	Work Package	Main Output	Status
1	Environment setup, data cleaning, EDA	Clean data, 12 EDA findings, charts	Completed
2	Preprocessing pipeline + leakage check	Reusable sklearn Pipeline	Completed
3	Train and compare models	Model comparison report	Completed
4	Tuning, calibration, final test	Registered model (AUC 0.952)	Completed
5	Decision bands and NPA simulation	Business simulation report	Completed
6	SHAP explainability	Global + local explanations	Completed
7	FastAPI backend + PostgreSQL	Secure scoring API, tests	Completed
8	Streamlit frontend	Officer pages + admin dashboard	In progress
9	Drift monitoring and retraining	Evidently/PSI reports, scheduler	Planned

Step	Work Package	Main Output	Status
10	Docker Compose, CI (GitHub Actions), README	One-command run, automated tests	Planned
11	Final report, presentation, viva preparation	Report + slides	Planned

9. Conclusion and Future Scope

The project shows that an explainable machine-learning system can predict loan default accurately (test ROC-AUC 0.952), reduce simulated NPAs far beyond the 30% target (75.8% fewer defaults) and still give a clear reason for every decision. Built fully in Python with FastAPI, Streamlit and PostgreSQL, it scores an application in about a tenth of a second and keeps a complete audit trail.

Future scope:

- **Real data:** test on a larger, real bank or bureau dataset with application dates for true drift monitoring.
- **Fairness checks:** measure and reduce bias across age and income groups.
- **More signals:** bank-statement and transaction data, alternative credit data for first-time borrowers.
- **Counterfactual advice:** tell rejected applicants what would change the decision (e.g. a smaller loan amount).
- **Cloud deployment:** HTTPS hosting with monitoring (Prometheus / Grafana) and automatic CI/CD.

10. Academic Review & Sign-Off

This synopsis has been submitted for review and approval of the proposed project scope, requirements and methodology by the project guide and the department.

Project Coordinator / Guide	Internal Examiner	Head of Department (HOD)
-----------------------------	-------------------	--------------------------

Signature: _____
Date: _____

Signature: _____
Date: _____

Signature: _____
Date: _____